📊 Grouping and Filtering Data in SQL – Notes

🔷 1. GROUP BY Clause

✅ **Purpose:**

- Used to **group rows** that have the same values in specific columns.
- Works with **aggregate functions**:
  - COUNT()
  - SUM()
  - AVG()
  - MIN()
  - MAX()

✅ **Basic Syntax:**

```
SELECT column1, aggregate_function(column2)
FROM table_name
WHERE condition
GROUP BY column1;
```

📦 **Dataset: products Table**

Columns:

- product_name
- brand_name
- brand_tag
- product_tag
- rating
- rating_count
- marked_price
- discounted_price
- discount_percent

🔷 2. Basic Grouping Queries

📌 **Unique Brand Names**

```
SELECT DISTINCT brand_name FROM products;
```

📌 **Count of Unique Brands**

```
SELECT COUNT(DISTINCT brand_name) FROM products;
```

📌 **Count Products in Each Brand**

```
SELECT brand_tag, COUNT(product_tag)
FROM products
GROUP BY brand_tag;
```

🔝 **3. Top Analysis Queries**

📌 **Top 5 Brands with Most Products**

```
SELECT brand_tag, COUNT(product_tag) AS products
FROM products
GROUP BY brand_tag
ORDER BY products DESC
LIMIT 5;
```

📌 **Top 5 Brands with Most Products Sold**

```
SELECT brand_tag, SUM(rating_count) AS products_sold
FROM products
GROUP BY brand_tag
ORDER BY products_sold DESC
LIMIT 5;
```

📌 **Top 5 Most Expensive Brands**

```
SELECT brand_tag, ROUND(AVG(discounted_price)) AS average_price
FROM products
GROUP BY brand_tag
ORDER BY average_price DESC
LIMIT 5;
```

📌 **Top 5 Least Expensive Brands**

```
ORDER BY average_price ASC;
```

🔷 4. Filtering Data

✅ WHERE Clause

- Filters rows **before grouping**
- Used with normal columns

Example:

```
SELECT *
FROM sales
WHERE product_id = 101;
```

✅ HAVING Clause

- Filters **after GROUP BY**
- Used with aggregate functions

Example:

```
SELECT product_id, SUM(quantity) AS total_quantity
FROM sales
GROUP BY product_id
HAVING SUM(quantity) > 100;
```

🔷 5. Category & Brand Analysis

📌 Top 10 Best-Selling Categories

```
SELECT product_tag, SUM(rating_count) AS product_sold
FROM products
GROUP BY product_tag
ORDER BY product_sold DESC
LIMIT 10;
```

📌 Top 5 Brands with Maximum Discount

```
SELECT brand_tag, AVG(discount_percent) AS avg_discount
FROM products
GROUP BY brand_tag
ORDER BY avg_discount DESC
LIMIT 5;
```

📌 **Most Expensive Categories**

```
SELECT product_tag, AVG(discounted_price) AS avg_price
FROM products
GROUP BY product_tag
ORDER BY avg_price DESC
LIMIT 5;
```

🔷 **6. Combining WHERE + GROUP BY + HAVING**

📌 **Brand Report Card**

```
SELECT brand_tag,
    SUM(rating_count) AS people_rated,
    MIN(marked_price) AS min_price,
    AVG(marked_price) AS avg_price,
    MAX(marked_price) AS max_price
FROM products
GROUP BY brand_tag;
```

🔷 **7. Category-Specific Filtering**

Example: Top 5 Brands Selling T-Shirts

```
SELECT brand_tag,
    SUM(rating_count) AS orders,
    AVG(discounted_price) AS avg_price
FROM products
WHERE product_tag = 'tshirts'
GROUP BY brand_tag
ORDER BY orders DESC
LIMIT 5;
```

(Same logic for shirts, jeans, dresses)

🔷 **8. Advanced Grouping**

📌 **Most Popular Product Names**

```
SELECT product_name, COUNT(*) AS name_count
FROM products
GROUP BY product_name
ORDER BY name_count DESC
LIMIT 10;
```

📌 **Products by Rating**

```
SELECT rating, COUNT(*) AS freq
FROM products
GROUP BY rating
ORDER BY rating ASC;
```

📌 **Rating Distribution for Nike**

```
SELECT rating, COUNT(*) AS freq
FROM products
WHERE brand_tag = 'nike'
GROUP BY rating;
```

🔷 **9. Price vs Rating Analysis (T-Shirts)**

```
SELECT rating,
    ROUND(AVG(rating_count)) AS avg_product_count,
    ROUND(AVG(discounted_price)) AS avg_price
FROM products
WHERE product_tag = 'tshirts'
GROUP BY rating
ORDER BY rating ASC;
```

🔷 **10. Category Rating Summary**

```
SELECT product_tag,
    AVG(rating) AS avg_rating,
    COUNT(*) AS total_products,
    SUM(rating_count) AS total_rating_count
FROM products
```

```
GROUP BY product_tag
ORDER BY avg_rating ASC;
```

🔷 11. High Price Brand Rating Analysis

```
SELECT brand_tag,
    AVG(rating) AS avg_rating,
    SUM(rating_count)
FROM products
WHERE discounted_price > 5000
GROUP BY brand_tag
ORDER BY avg_rating DESC;
```

🎯 Key Concepts Summary

Clause	Purpose
WHERE	Filters rows before grouping
GROUP BY	Groups similar values
HAVING	Filters grouped data
ORDER BY	Sorts results
LIMIT	Restricts number of rows
DISTINCT	Removes duplicates